PRE-ENTREGA 6

Analítica avanzada in-stream con Amazon Redshift

Documento técnico · Arquitectura · SQL · Evidencias de validación

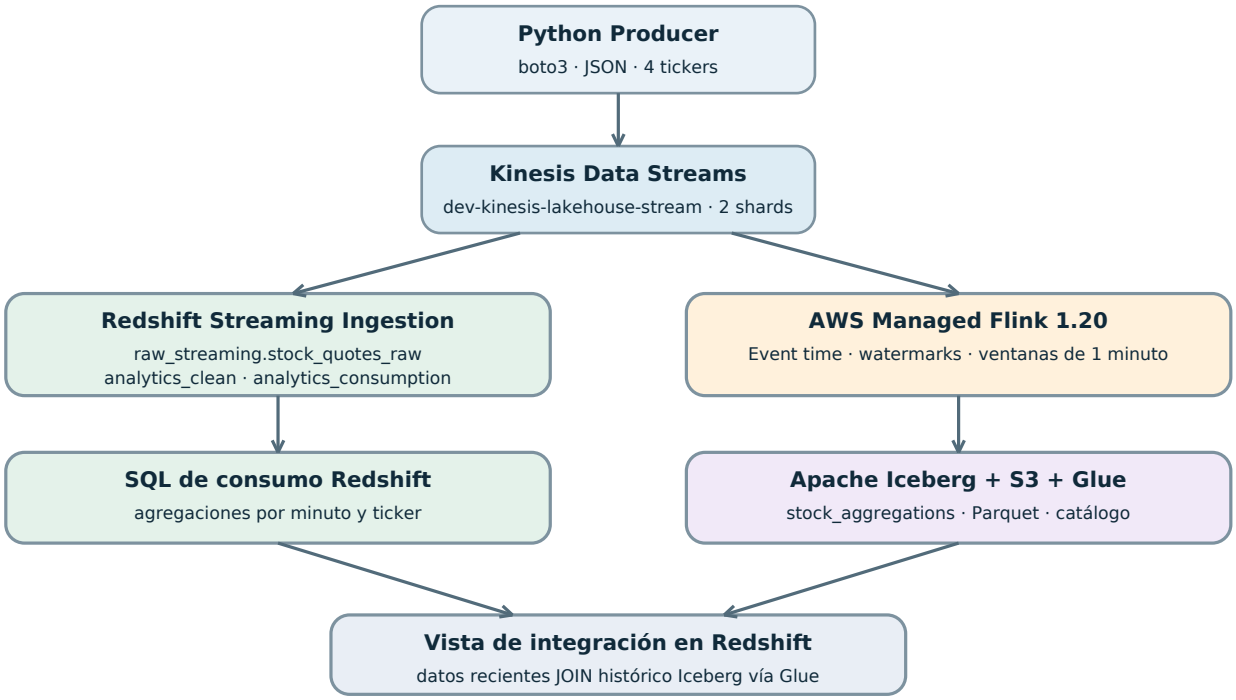
Proyecto	Lakehouse de cotizaciones bursátiles en tiempo real
Región AWS	us-east-2
Servicios principales	Kinesis Data Streams · Redshift · Managed Flink · Iceberg · Glue · S3
Infraestructura	Terraform
Estado	Pruebas ejecutadas; recursos posteriormente destruidos para evitar costos

Documento elaborado a partir de los scripts SQL y las capturas de evidencia del proyecto.

1. Objetivo

Extender el pipeline de cotizaciones bursátiles con una capa de analítica in-stream en Amazon Redshift. Un único flujo de Amazon Kinesis Data Streams alimenta dos ramas paralelas: una orientada a consultas de baja latencia y otra a la persistencia histórica mediante Flink e Iceberg.

2. Arquitectura



La rama de Redshift consume directamente Kinesis mediante Streaming Ingestion; S3 no actúa como intermediario de esa ruta. En paralelo, Flink procesa eventos y escribe agregaciones históricas en Iceberg, cuyos datos se almacenan en S3 y cuyos metadatos se administran mediante Glue. Redshift consulta el catálogo para integrar ambas escalas temporales.

3. Componentes y configuración

Componente	Configuración / función
Región	us-east-2
Kinesis	dev-kinesis-lakehouse-stream · 2 shards · retención 48 h
Productor	Python + boto3 · 20 eventos/lote · intervalo 2 s
Eventos	TICKER · PRICE · EVENT_TIME
Símbolos	AAPL · MSFT · GOOGL · AMZN
Redshift	redshift-lakehouse-dev · base analytics
Glue / Iceberg	stock_lakehouse_dev · stock_aggregations
Flink	Managed Flink 1.20 · ventanas de 1 minuto · agrupación por ticker
Formato histórico	Parquet en S3 · partición days(window_start)

4. Flujo de procesamiento

4.1 Productor y Kinesis

El productor Python genera cotizaciones sintéticas para cuatro tickers. Cada evento incluye símbolo, precio y timestamp UTC. Los eventos se envían mediante put_records, utilizando el ticker como partition key.

4.2 Rama histórica: Flink e Iceberg

Flink deserializa el JSON, utiliza event time y watermarks, agrupa por ticker y calcula por ventanas tumbling de un minuto la cantidad de operaciones (trade_count), precio promedio (avg_price), máximo (max_price) y mínimo (min_price). Los resultados se escriben en stock_aggregations, una tabla Iceberg administrada por Glue y almacenada como Parquet en S3.

4.3 Rama de tiempo real: Redshift Streaming Ingestion

Redshift se conecta directamente al stream mediante el esquema externo kds. La vista materializada raw_streaming.stock_quotes_raw conserva el payload como SUPER junto con metadatos de Kinesis y utiliza AUTO REFRESH YES.

4.4 Capas SQL

• Raw: conserva el JSON completo y metadatos del stream.

• Clean: extrae TICKER, EVENT_TIME y PRICE, convierte tipos y elimina duplicados por ticker, timestamp y precio.

• Consumption: agrupa por minuto y ticker, calcula cantidad de registros y suma de precios; el promedio se calcula como suma / cantidad.

La vista materializada de consumo se actualiza manualmente mediante REFRESH MATERIALIZED VIEW, de acuerdo con el script y la validación realizada.

4.5 Integración con el histórico

Redshift crea el esquema externo lake apuntando a stock_lakehouse_dev en Glue. La vista stock_quotes_vs_historico combina el precio promedio reciente con el promedio histórico de lake.stock_aggregations y calcula el desvío.

5. Scripts SQL del proyecto

Se incluyen los seis scripts recuperados del archivo SQL del proyecto. Los bloques reproducen el contenido fuente; se mantienen los nombres de objetos y las referencias configuradas para el laboratorio.

01_ingest.sql — Ingesta desde Kinesis y streaming materialized view

```
-- ============================================================
-- PRE-ENTREGA 6 - REDSHIFT STREAMING INGESTION
-- Paso 1: Kinesis -> Streaming Materialized View
-- Región: us-east-2
-- ============================================================

-- 1.1 Esquema externo conectado a Kinesis
-- Reemplazar el ARN por el output real de Terraform.

CREATE EXTERNAL SCHEMA IF NOT EXISTS kds
FROM KINESIS
REGION 'us-east-2'
IAM_ROLE 'arn:aws:iam::286123979983:role/stock-lakehouse-redshift-dev';

-- 1.2 Esquemas por capa

CREATE SCHEMA IF NOT EXISTS raw_streaming;

CREATE SCHEMA IF NOT EXISTS analytics_clean;

CREATE SCHEMA IF NOT EXISTS analytics_consumption;

-- 1.3 Materialized View de ingesta
-- Conserva el JSON completo en formato SUPER.
-- Los registros que no sean JSON válido se descartan.

CREATE MATERIALIZED VIEW raw_streaming.stock_quotes_raw
AUTO REFRESH YES
AS
SELECT
    approximate_arrival_timestamp,
    partition_key,
    shard_id,
    sequence_number,
    refresh_time,
    JSON_PARSE(kinesis_data) AS payload
FROM kds."dev-kinesis-lakehouse-stream"
WHERE CAN_JSON_PARSE(kinesis_data);

-- 1.4 Primer refresh manual

REFRESH MATERIALIZED VIEW raw_streaming.stock_quotes_raw;

-- 1.5 Verificación de ingesta

SELECT
    COUNT(*) AS registros,
    MIN(approximate_arrival_timestamp) AS desde,
    MAX(approximate_arrival_timestamp) AS hasta
FROM raw_streaming.stock_quotes_raw;

-- 1.6 Inspección de registros

SELECT *
FROM raw_streaming.stock_quotes_raw
LIMIT 10;
```

02_clean_layer.sql — Capa limpia: extracción, tipado y deduplicación

```
-- PRE-ENTREGA 6 - Paso 2: tipado y deduplicacion

CREATE OR REPLACE VIEW analytics_clean.stock_quotes_unicos AS
WITH datos_extraidos AS (
    SELECT
        JSON_EXTRACT_PATH_TEXT(
            JSON_SERIALIZE(payload), 'TICKER'
        ) AS ticker,

        JSON_EXTRACT_PATH_TEXT(
            JSON_SERIALIZE(payload), 'EVENT_TIME'
        )::TIMESTAMPTZ AS event_time,

        JSON_EXTRACT_PATH_TEXT(
            JSON_SERIALIZE(payload), 'PRICE'
        )::DECIMAL(18,6) AS price,

        approximate_arrival_timestamp,
        sequence_number

    FROM raw_streaming.stock_quotes_raw
),
datos_numerados AS (
    SELECT
        ticker,
        event_time,
        price,
        approximate_arrival_timestamp,

        ROW_NUMBER() OVER (
            PARTITION BY ticker, event_time, price
            ORDER BY approximate_arrival_timestamp, sequence_number
        ) AS rn

    FROM datos_extraidos
)
SELECT
    ticker,
    event_time,
    price,
    approximate_arrival_timestamp
FROM datos_numerados
WHERE rn = 1;

-- Verificacion
SELECT *
FROM analytics_clean.stock_quotes_unicos
LIMIT 20;
```

03_consumer.sql — Capa de consumo: agregaciones por minuto y ticker

```
-- PRE-ENTREGA 6 - Paso 3: capa de consumo
-- Agregaciones por minuto y ticker.
-- Actualizacion manual de la vista materializada.

CREATE MATERIALIZED VIEW analytics_consumption.stock_quotes_por_minuto
AS
SELECT
    DATE_TRUNC(
        'minute',
        JSON_EXTRACT_PATH_TEXT(
            JSON_SERIALIZE(payload), 'EVENT_TIME'
        )::TIMESTAMPTZ
    ) AS minuto,

    JSON_EXTRACT_PATH_TEXT(
        JSON_SERIALIZE(payload), 'TICKER'
    ) AS ticker,

    COUNT(*) AS registros,

    SUM(
        JSON_EXTRACT_PATH_TEXT(
            JSON_SERIALIZE(payload), 'PRICE'
        )::DECIMAL(18,6)
    ) AS suma_precios

FROM raw_streaming.stock_quotes_raw
GROUP BY 1, 2;

-- Actualizacion manual
REFRESH MATERIALIZED VIEW
analytics_consumption.stock_quotes_por_minuto;

-- Consulta del tablero: cotizaciones de los ultimos 5 minutos.
SELECT
    minuto,
    ticker,
    registros,
    suma_precios,
    ROUND(suma_precios / NULLIF(registros, 0), 4) AS precio_promedio
FROM analytics_consumption.stock_quotes_por_minuto
WHERE minuto >= DATEADD(minute, -5, GETDATE())
ORDER BY minuto DESC, ticker;
```

04_lakehouse.sql — Integración de resultados recientes con el histórico Iceberg

```
-- PRE-ENTREGA 6 - Paso 4: integracion con el Lakehouse.
-- Este esquema externo apunta al Glue Data Catalog de la Pre-entrega 5.

CREATE EXTERNAL SCHEMA IF NOT EXISTS lake
FROM DATA CATALOG
DATABASE 'stock_lakehouse_dev'
REGION 'us-east-2'
IAM_ROLE 'arn:aws:iam::286123979983:role/stock-lakehouse-redshift-dev';

-- La streaming MV no admite JOIN.
-- La integracion con el historico se realiza en una vista superior.

CREATE OR REPLACE VIEW analytics_consumption.stock_quotes_vs_historico AS
SELECT
    c.ticker,
    c.minuto,
    c.registros,
    c.precio_promedio,
    h.precio_promedio_historico,
    c.precio_promedio - h.precio_promedio_historico AS desvio
FROM (
    SELECT
        ticker,
        minuto,
        registros,
        suma_precios / NULLIF(registros, 0) AS precio_promedio
    FROM analytics_consumption.stock_quotes_por_minuto
) c
JOIN (
    SELECT
        ticker,
        AVG(avg_price) AS precio_promedio_historico
    FROM lake.stock_aggregations
    GROUP BY ticker
) h
    ON h.ticker = c.ticker;

-- Verificacion de la integracion.
SELECT *
FROM analytics_consumption.stock_quotes_vs_historico
ORDER BY minuto DESC, ticker
LIMIT 20;
```

05_security.sql — Seguridad y control de acceso basado en roles

```
-- PRE-ENTREGA 6 - Paso 5: RBAC con ROLES.

CREATE ROLE analytics_rt;
CREATE ROLE analytics_ar;

-- La capa raw queda restringida.
REVOKE ALL ON SCHEMA raw_streaming FROM PUBLIC;

-- El rol de tiempo real puede consultar la capa de consumo.
GRANT USAGE ON SCHEMA analytics_consumption TO ROLE analytics_rt;
GRANT SELECT ON ALL TABLES IN SCHEMA analytics_consumption TO ROLE analytics_rt;

-- El rol de analitica tambien necesita acceso al esquema.
GRANT USAGE ON SCHEMA analytics_consumption TO ROLE analytics_ar;
```

06_monitoring.sql — Monitoreo de refresh, lectura del stream, errores y frescura

```
-- PRE-ENTREGA 6 - Paso 6: monitoreo.
-- Consultas sobre las vistas SYS de Amazon Redshift.

-- 6.1 Salud de los refresh de las vistas materializadas.
SELECT
    mv_name,
    refresh_type,
    status,
    start_time,
    end_time,
    duration
FROM sys_mv_refresh_history
ORDER BY start_time DESC
LIMIT 20;


-- 6.2 Estado de lectura del stream por shard.
-- El lag se estima comparando el momento del lote
-- con la llegada del ultimo registro leido.

SELECT
    partition_id,
    MAX(record_time) AS ultimo_lote,
    SUM(scanned_rows) AS filas,
    DATEDIFF(
        second,
        MAX(stream_record_time_max),
        MAX(record_time)
    ) AS lag_aprox_seg
FROM sys_stream_scan_states
WHERE mv_name = 'stock_quotes_raw'
  AND record_time > DATEADD(hour, -1, GETDATE())
GROUP BY partition_id
ORDER BY partition_id;


-- 6.3 Registros con errores de ingesta.
-- SYS_STREAM_SCAN_ERRORS registra, entre otros casos,
-- errores por exceder el tamano maximo permitido.

SELECT *
FROM sys_stream_scan_errors
WHERE mv_name = 'stock_quotes_raw'
ORDER BY record_time DESC
LIMIT 20;


-- 6.4 Frescura de punta a punta:
-- segundos transcurridos desde la llegada del ultimo registro.

SELECT
    DATEDIFF(
        second,
        MAX(approximate_arrival_timestamp),
        GETDATE()
    ) AS segundos_desde_ultimo_registro
FROM raw_streaming.stock_quotes_raw;
```

6. Seguridad y monitoreo

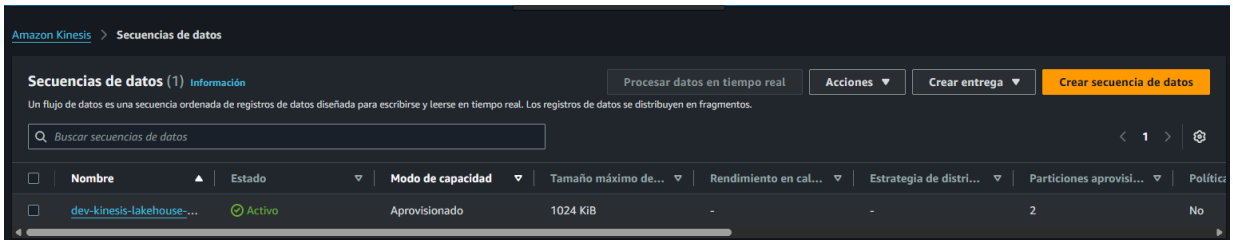
Los scripts de seguridad crean los roles analytics_rt y analytics_ar. analytics_rt recibe USAGE sobre analytics_consumption y SELECT sobre sus tablas; analytics_ar recibe USAGE sobre el esquema. Se revoca el acceso PUBLIC a raw_streaming.

El monitoreo utiliza SYS_MV_REFRESH_HISTORY, SYS_STREAM_SCAN_STATES y SYS_STREAM_SCAN_ERRORS. El lag aproximado por shard se estima comparando el momento del lote con el timestamp del último registro leído. La frescura se calcula a partir de la llegada del último registro disponible en la vista raw.

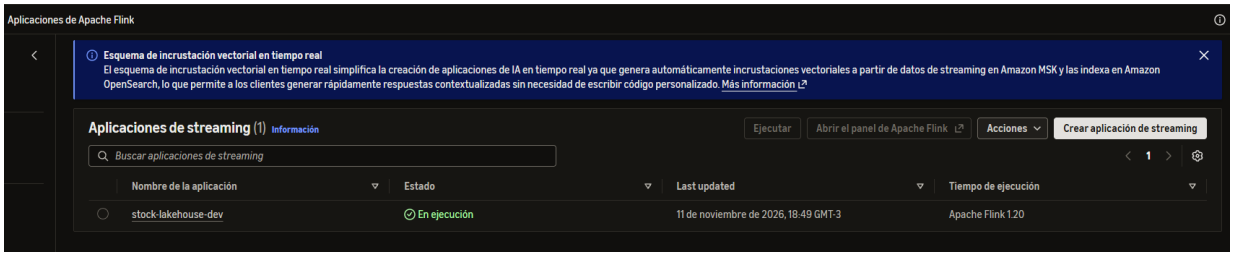
7. Evidencias de ejecución

Las capturas originales se presentan a continuación agrupadas por etapa. Se preservan los nombres de archivo entregados en evidencia.zip.

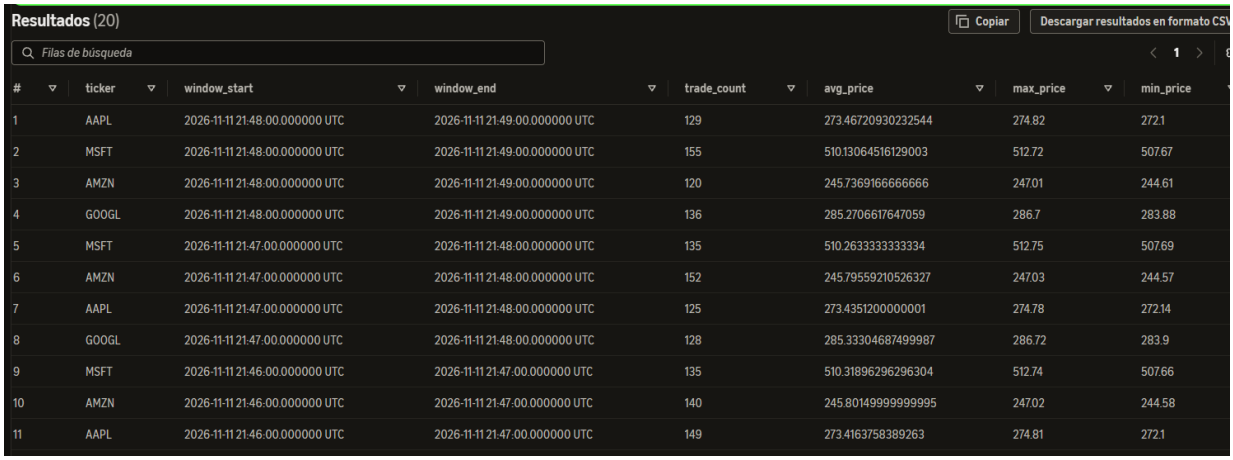
7.1 Infraestructura y procesamiento



00 kinesis running

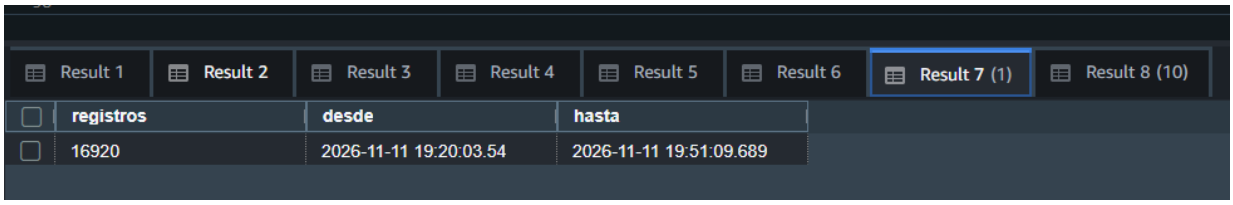


00 flink running



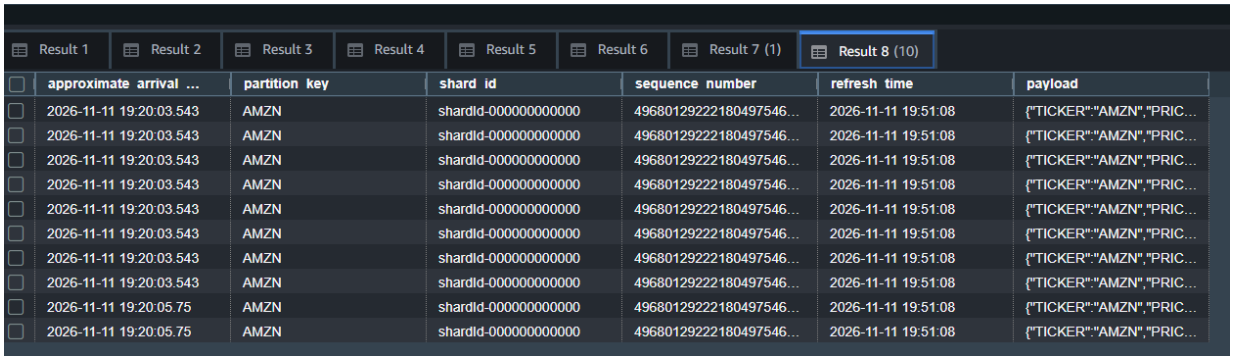
00 kinesis flink iceberg

7.2 Ingesta y capas de Redshift



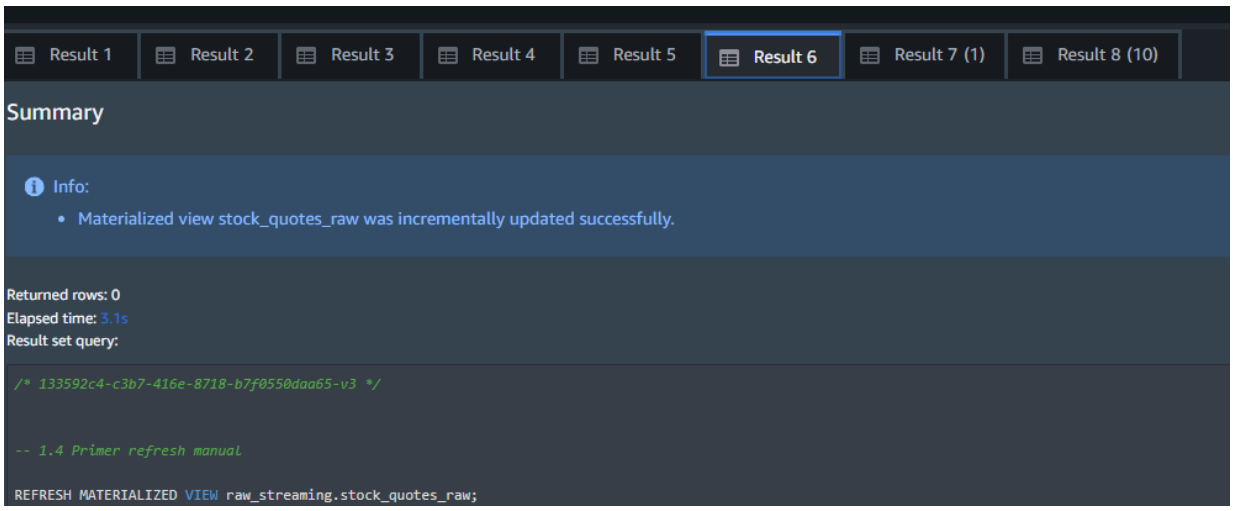
registros	desde	hasta
16920	2026-11-11 19:20:03.54	2026-11-11 19:51:09.689

01 ingest regis



approximate_arrival_...	partition_key	shard_id	sequence_number	refresh_time	payload
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:03.543	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:05.75	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...
2026-11-11 19:20:05.75	AMZN	shardId-000000000000	49680129222180497546...	2026-11-11 19:51:08	{"TICKER":"AMZN","PRIC...

01 ingest raw



Summary

Info:

- Materialized view stock_quotes_raw was incrementally updated successfully.

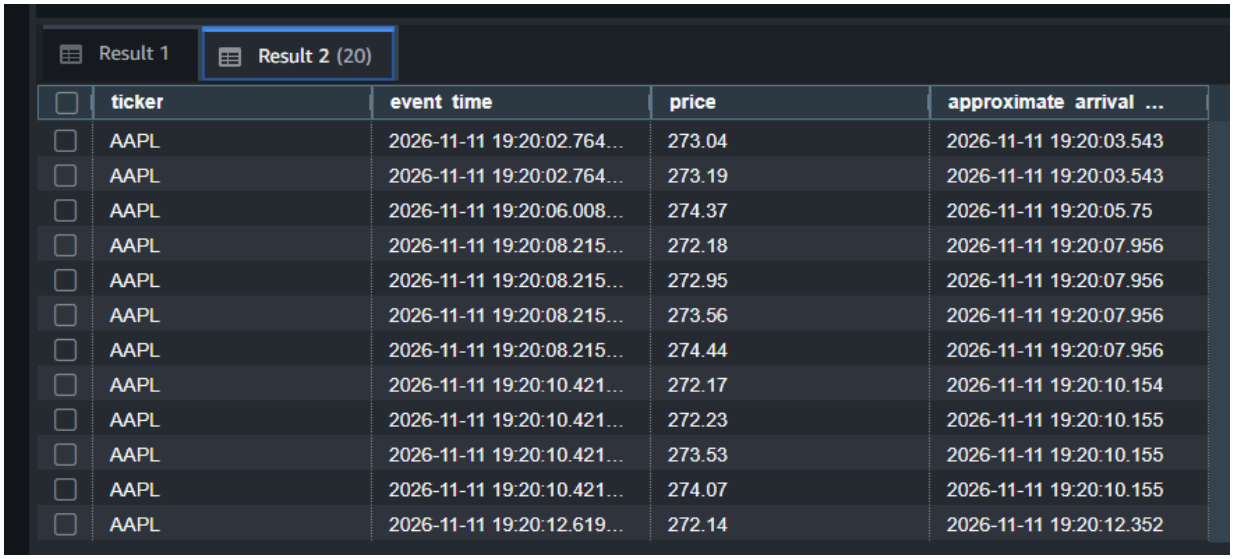
Returned rows: 0
Elapsed time: 3.1s
Result set query:

```
/* 133592c4-c3b7-416e-8718-b7f0550daa65-v3 */


-- 1.4 Primer refresh manual

REFRESH MATERIALIZED VIEW raw_streaming.stock_quotes_raw;
```

01 ingest mview



ticker	event_time	price	approximate_arrival_...
AAPL	2026-11-11 19:20:02.764...	273.04	2026-11-11 19:20:03.543
AAPL	2026-11-11 19:20:02.764...	273.19	2026-11-11 19:20:03.543
AAPL	2026-11-11 19:20:06.008...	274.37	2026-11-11 19:20:05.75
AAPL	2026-11-11 19:20:08.215...	272.18	2026-11-11 19:20:07.956
AAPL	2026-11-11 19:20:08.215...	272.95	2026-11-11 19:20:07.956
AAPL	2026-11-11 19:20:08.215...	273.56	2026-11-11 19:20:07.956
AAPL	2026-11-11 19:20:08.215...	274.44	2026-11-11 19:20:07.956
AAPL	2026-11-11 19:20:10.421...	272.17	2026-11-11 19:20:10.154
AAPL	2026-11-11 19:20:10.421...	272.23	2026-11-11 19:20:10.155
AAPL	2026-11-11 19:20:10.421...	273.53	2026-11-11 19:20:10.155
AAPL	2026-11-11 19:20:10.421...	274.07	2026-11-11 19:20:10.155
AAPL	2026-11-11 19:20:12.619...	272.14	2026-11-11 19:20:12.352

02 clean layer structured

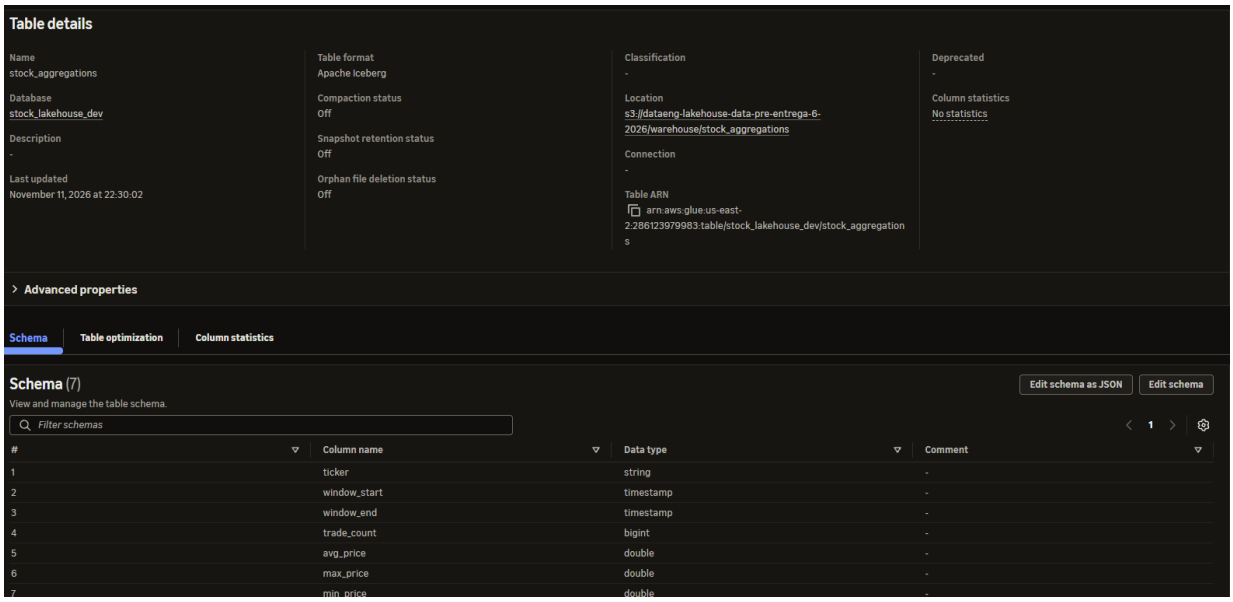


Result 1 | Result 2 | Result 3 (20)

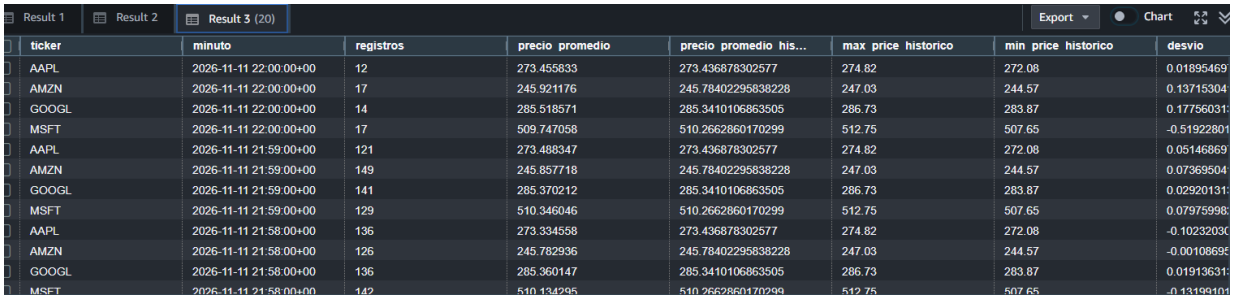
minuto	ticker	registros	suma_precios	precio_promedio
2026-11-11 20:12:00+00	AAPL	37	10114.59	273.3673
2026-11-11 20:12:00+00	AMZN	37	9091.39	245.7132
2026-11-11 20:12:00+00	GOOGL	37	10550.04	285.1362
2026-11-11 20:12:00+00	MSFT	29	14805.65	510.5397
2026-11-11 20:11:00+00	AAPL	139	38016.45	273.4996
2026-11-11 20:11:00+00	AMZN	151	37120	245.8278
2026-11-11 20:11:00+00	GOOGL	133	37957.89	285.3977
2026-11-11 20:11:00+00	MSFT	137	69883.85	510.1011
2026-11-11 20:10:00+00	AAPL	128	35003.49	273.4648
2026-11-11 20:10:00+00	AMZN	138	33928.72	245.8603
2026-11-11 20:10:00+00	GOOGL	125	35664.07	285.3126
2026-11-11 20:10:00+00	MSFT	149	76054.09	510.4301

03 consumer sum avg

7.3 Iceberg e integración

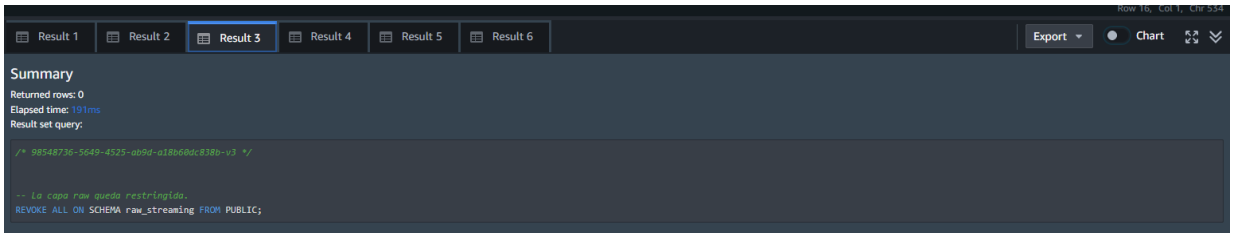


04 iceberg glue metadata

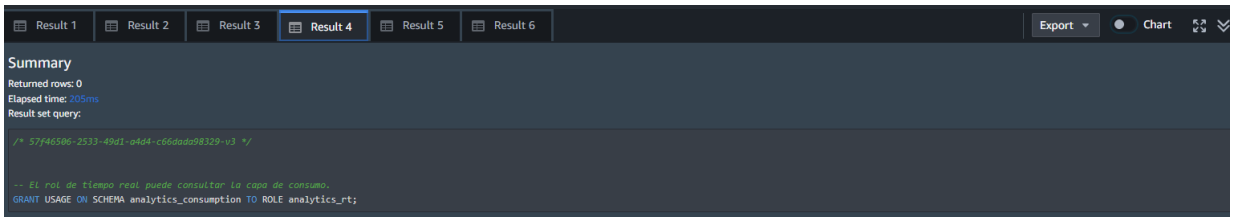


04 lakehouse shift iceberg join

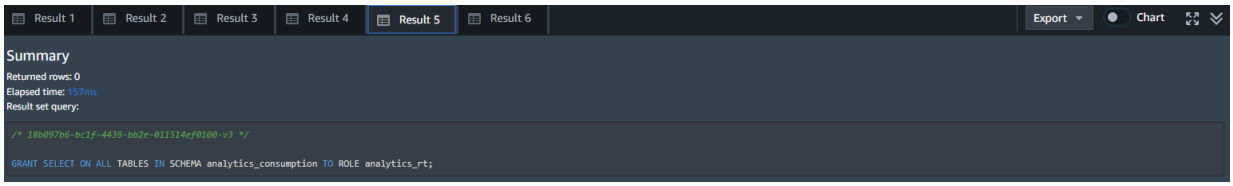
7.4 Seguridad y permisos



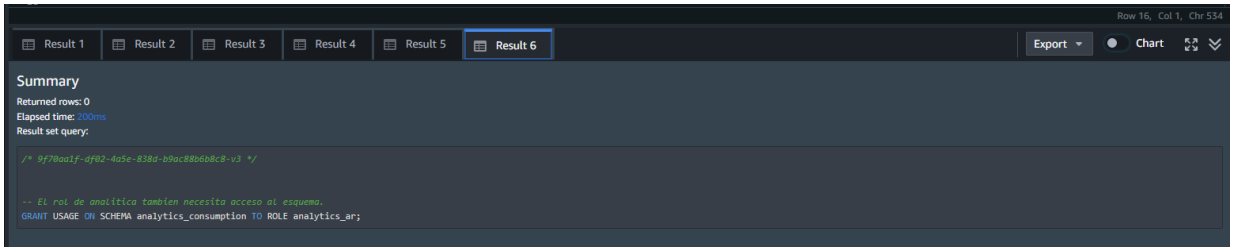
05 security revoke public



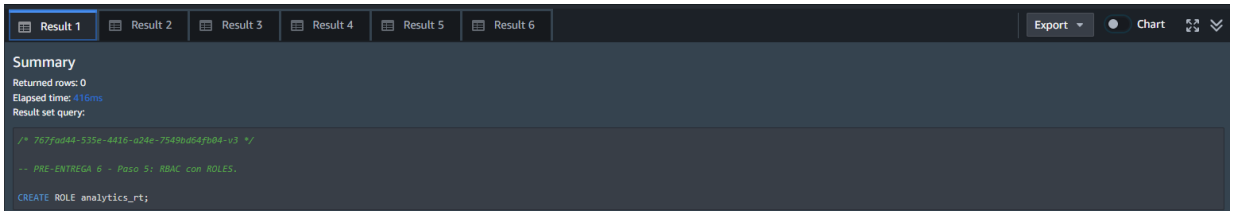
05 security grant usage rt



05 security grant table rt



05 security grant usage ar

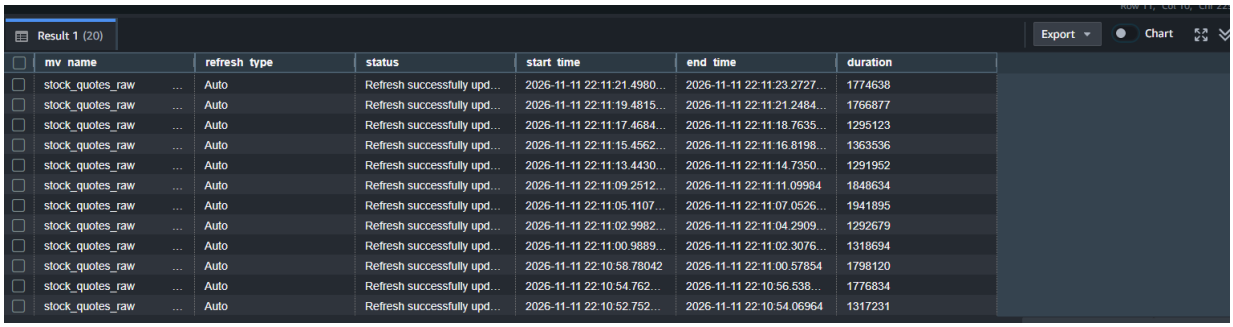


05 security analytics rt



05 security analytics ar

7.5 Monitoreo

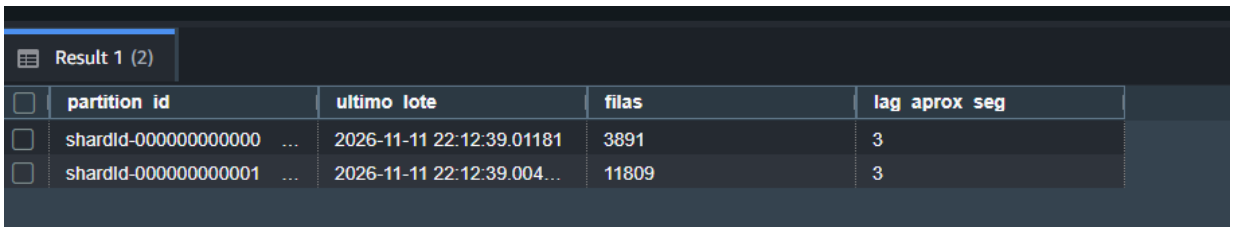


Result 1 (20)

Export · Chart

mv_name	refresh_type	status	start_time	end_time	duration
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:21.4980...	2026-11-11 22:11:23.2727...	1774638
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:19.4815...	2026-11-11 22:11:21.2484...	1766877
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:17.4684...	2026-11-11 22:11:18.7635...	1295123
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:15.4562...	2026-11-11 22:11:16.8198...	1363536
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:13.4430...	2026-11-11 22:11:14.7350...	1291952
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:09.2512...	2026-11-11 22:11:11.09984	1848634
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:05.1107...	2026-11-11 22:11:07.0526...	1941895
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:02.9982...	2026-11-11 22:11:04.2909...	1292679
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:11:00.9889...	2026-11-11 22:11:02.3076...	1318694
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:10:58.78042	2026-11-11 22:11:00.57854	1798120
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:10:54.762...	2026-11-11 22:10:56.538...	1776834
stock_quotes_raw	Auto	Refresh successfully upd...	2026-11-11 22:10:52.752...	2026-11-11 22:10:54.06964	1317231

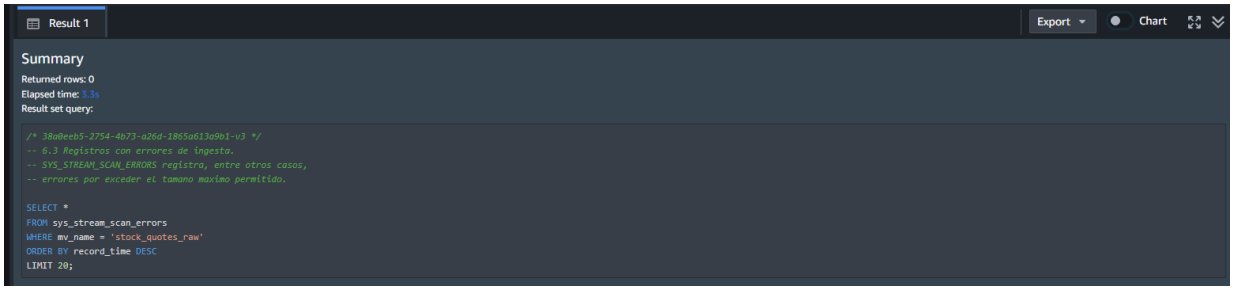
06 monitoring history refresh



Result 1 (2)

partition_id	ultimo_lote	filas	lag_aprox_seg
shardId-000000000000 ...	2026-11-11 22:12:39.01181	3891	3
shardId-000000000001 ...	2026-11-11 22:12:39.004...	11809	3

06 monitoring latency



Result 1

Export · Chart

Summary

Returned rows: 0

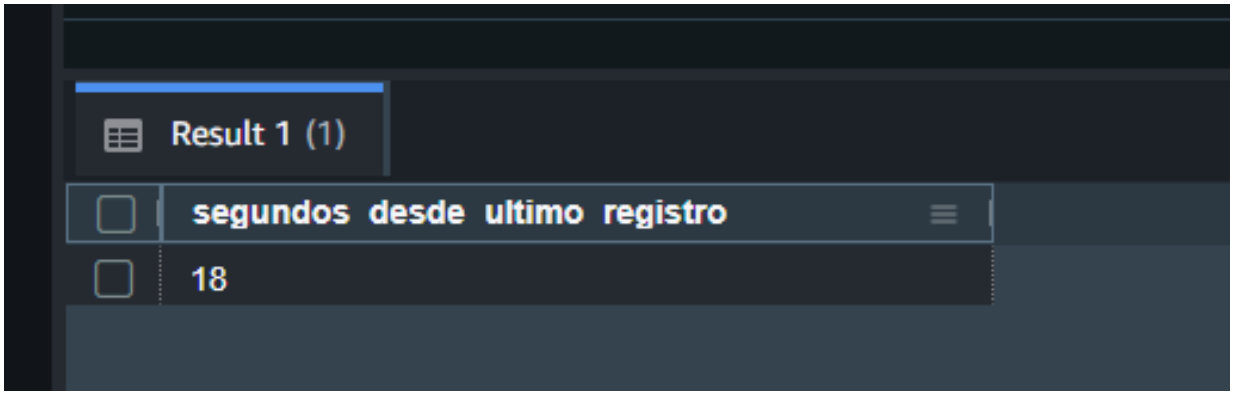
Elapsed time: 3.3s

Result set query:

```
/* 38a0eeb5-2754-4b73-a26d-1865a613a9b1-v3 */
-- 6.3 Registros con errores de ingesta.
-- SYS_STREAM_SCAN_ERRORS registra, entre otros casos,
-- errores por exceder el tamano maximo permitido.

SELECT *
FROM sys_stream_scan_errors
WHERE mv_name = 'stock_quotes_raw'
ORDER BY record_time DESC
LIMIT 20;
```

06 monitoring no errors



Result 1 (1)

segundos_desde_ultimo_registro
18

06 monitoring data freshness

8. Resultados y consideraciones

La evidencia permite verificar la recepción de eventos en Kinesis, la ejecución de Flink, la disponibilidad de registros en Redshift, las consultas de agregación y la integración con la tabla Iceberg registrada en Glue. La consulta de integración expone valores recientes junto con el promedio histórico y el desvío calculado.

La solución separa dos necesidades analíticas: acceso de baja latencia mediante Redshift Streaming Ingestion y persistencia histórica mediante Iceberg. La separación por capas SQL facilita inspeccionar el dato original, aplicar transformaciones y exponer resultados de consumo.

9. Estado de la infraestructura

La infraestructura se administró con Terraform. Una vez realizadas las pruebas y capturadas las evidencias, los recursos se destruyeron para evitar costos recurrentes. Este documento registra una implementación validada durante el laboratorio; no representa un entorno actualmente activo.

10. Estructura relevante del repositorio

```
app/          Aplicación Java de Flink
infra/        Recursos AWS definidos con Terraform
producer/     Productor Python de cotizaciones
sql/          Scripts SQL de ingesta, limpieza, consumo,
              integración con Iceberg, seguridad y monitoreo
evidencia/    Capturas de validación
README.md     Documentación de ejecución
```